IC-272 — Introduction to Machine Learning

Lab 9 — Clustering with K-Means

Indian Institute of Technology Mandi

Total marks: 100 — Duration: 2 hours

Marks: 20 for code, 80 for the viva. There are three tasks. Your TA takes a short viva on each task *as soon as you finish it* — do not save them all for the end.

There is also a homework section at the end. It will not be vivaed and carries no marks today. It is examinable in the final lab exam.

You will reuse `pairwise_distances` from Lab 1 — it is the heart of today's algorithm. Copy it into a cell at the top.

Rules

- Permitted: `numpy`, `pandas`, `matplotlib`. **`scikit-learn` and `scipy` are not permitted** — a task that calls either scores zero for that task.
- Every function named in this handout must appear with exactly that name.

Setting up — type this first

`ic272_lab9_customers.csv` holds the locations of 300 customers of a delivery company, in kilometres on a city map. **There is no label column.** Nobody has told us which customers belong together; finding that out is the job.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("ic272_lab9_customers.csv")
X = df[["x_km", "y_km"]].to_numpy(float)
print(X.shape)                                    # (300, 2)

def random_init(X, K, seed):
    """K distinct data points, chosen at random, as starting centroids."""
    idx = np.random.default_rng(seed).choice(len(X), K, replace=False)
    return X[idx]
```

`random_init` fixes how everyone starts, so that your numbers can be compared with your neighbour's. Use it exactly as written.

Why this lab exists — read this, then run it

Plot the data:

```
fig, ax = plt.subplots()
ax.scatter(X[:, 0], X[:, 1], s=14)
ax.set_xlabel("x (km)"); ax.set_ylabel("y (km)"); ax.set_aspect("equal")
```

You will see groups of customers by eye. Suppose the company wants to build one depot per group. Where should the depots go, and which customers should each serve? There is no right answer to check against — no label column, no accuracy. That is *unsupervised* learning.

This is not K-NN

Both have a k and both use distances, so it is worth being clear:

	k-NN (Lab 6)	**K-means (today)**
needs labels?	yes — supervised	no — unsupervised
what k means	neighbours that vote	clusters to find
what it produces	a prediction for a new point	a grouping of the data you have
how k is chosen	cross-validation on accuracy	the elbow method — no accuracy exists

The algorithm, from the lecture

Pick K starting centroids, then repeat two steps until nothing changes:

1. **Assign:** give each point to its nearest centroid.
2. **Update:** move each centroid to the mean of the points assigned to it.

The quality of a clustering is measured by the *distortion*

$$J = \sum_{i=1}^{m} \left\| x^{(i)} - \mu_{c^{(i)}} \right\|^2,$$

the total squared distance from every point to the centroid of its own cluster. Each step can only lower J or leave it unchanged, so the algorithm always stops — but, as you will see, not necessarily at the best answer.

Task 1 — K-means from scratch [code 8 — viva 32]

(a) Write `assign_clusters(X, centroids)` returning, for every point, the index of its nearest centroid. Use your Lab 1 `pairwise_distances`. **[2]**

(b) Write `update_centroids(X, labels, centroids)` returning the new centroids, each the mean of its assigned points. If a cluster has no points, keep its old centroid. **[2]**

(c) Write `distortion(X, labels, centroids)` returning J. **[1]**

(d) Write `kmeans(X, init_centroids, max_iter=100)` that alternates (a) and (b) until the centroids stop changing (`np.allclose`), and returns the final centroids, the labels, J, and the number of iterations used. **[3]**

Check as you go on the lecture's own example: the 15 points A1–A15 from the slides, $K = 3$, starting centroids $(2, 6)$, $(5, 10)$, $(6, 11)$. Your centroids after each iteration must match the slides, and the algorithm must stop with centroids (4, 4.6), (4.143, 9.571), (10, 11.333). Get this right before calling your TA — Tasks 2 and 3 are built on this function.

Task 2 — Where you start matters [code 6 — viva 24]

The lecture says K-means is "prone to the local minimum". Test it on the customer data.

(a) Run `kmeans` with $K = 6$ ten times, starting from `random_init(X, 6, seed)` for `seed` $= 0, 1, \ldots, 9$. Print J and the number of iterations for each. **[3]**

(b) Plot the clustering with the lowest J and the one with the highest J side by side, colouring points by cluster and marking the centroids. **[3]**

Look at the worst one and say, in terms of where the six starting centroids happened to fall, how it got stuck there — and why the update step could never get it out.

Task 3 — Choosing K with the elbow method [code 6 — viva 24]

There is no accuracy to cross-validate, so how many depots should the company build?

(a) Write `best_of_kmeans(X, K, n_init=10)` that runs `kmeans` from `random_init` with seeds 0 to `n_init` − 1 and returns the result with the lowest J. **[3]**

(b) For $K = 1, 2, \ldots, 10$, record the best J and plot J against K. **[2]**

(c) Report the K at the elbow. **[1]**

J keeps falling as K grows, all the way to $J = 0$ when every point is its own cluster. So "pick the K with the smallest J" cannot be the rule. Before calling your TA, be ready to say what the elbow is actually looking for. Task 2 is also why (a) takes the best of several starts rather than one.

Homework — not vivaed today, examinable in the final lab exam

Complete this homework before the next lab. The functions you write here are reused later in the course and will be called by the names given below. If you skip them now, you will be writing them under time pressure later.

H1. K-means++. Write `kmeans_pp_init(X, K, seed)` following the lecture: pick the first centroid uniformly at random, then pick each next one with probability proportional to $D(x)^2$, the squared distance from x to its nearest centroid chosen so far. Draw every random choice from one generator, `np.random.default_rng(seed)`. At $K = 6$, run `kmeans` from 100 random starts and from 100 K-means++ starts (seeds 0–99), and count how often each gets stuck at a J more than 5% above the best. Does K-means++ remove the problem, or only reduce it?

H2. Outliers. Add three points near $(30, 26)$ to the customer data and rerun `best_of_kmeans` with $K = 6$. Print the cluster sizes. What did one centroid spend itself on, and what happened to two of the real neighbourhoods as a result? Connect this to the lecture's "sensitive to outliers" point.

H3. Scale, again. Cluster the Lab 8 loan data (`income_k` and `utilisation` only) into $K = 2$ groups, once on the raw columns and once after standardising. For each cluster report its size, mean income, mean utilisation and default rate (the label is used only to describe the clusters, never to form them). Which feature decided the raw split, and why? Which version found groups that differ in risk?

H4. Image segmentation. Load `ic272_lab9_image.png` with `plt.imread` and keep only the first three channels, `img[:, :, :3]`. Reshape it to one row per pixel and three columns (red, green, blue), cluster the pixels for $K = 2, 3, 4$, replace every pixel by its centroid's colour, and show the three results beside the original. The picture has four regions. At which K does each appear, and why does the elbow on this data suggest a K that misses one of them?

Submission

- Call your TA as you finish each task. All three vivas must be done before you leave.
- Keep the notebook; the homework goes in the same file.